MO-SMAC: Multiobjective Sequential Model-Based Algorithm Configuration

Jeroen G. Rook j.g.rook@utwente.nl
Data Management & Biometrics, University of Twente, The Netherlands

Carolin Benjamins c.benjamins@ai.uni-hannover.de
Institute of Artificial Intelligence, Leibniz University Hannover, Germany

Jakob Bossek jakob.bossek@uni-paderborn.de
Machine Learning and Optimisation, Paderborn University, Germany

Heike Trautmann heike.trautmann@uni-paderborn.de
Machine Learning and Optimisation, Paderborn University, Germany; Data Management & Biometrics, University of Twente, The Netherlands

Holger H. Hoos hh@aim.rwth-aachen.de
Artificial Intelligence Methodology, RTWH Aachen University, Germany; ADA Research Group, Leiden University, The Netherlands; University of British Columbia, Canada

Marius Lindauer m.lindauer@ai.uni-hannover.de
Institute of Artificial Intelligence, L3S Research Center Leibniz University Hannover, Germany

https://doi.org/10.1162/evco_a_00371

Abstract

Automated algorithm configuration aims at finding well-performing parameter configurations for a given problem, and it has proven to be effective within many AI domains, including evolutionary computation. Initially, the focus was on excelling in one performance objective, but, in reality, most tasks have a variety of (conflicting) objectives. The surging demand for trustworthy and resource-efficient AI systems makes this multiobjective perspective even more prevalent. We propose a new general-purpose multiobjective automated algorithm configurator by extending the widely-used SMAC framework. Instead of finding a single configuration, we search for a nondominated set that approximates the actual Pareto set. We propose a pure multiobjective Bayesian optimization approach for obtaining promising configurations by using the predicted hypervolume improvement as acquisition function. We also present a novel intensification procedure to efficiently handle the selection of configurations in a multiobjective context. Our approach is empirically validated and compared across various configuration scenarios in four AI domains, demonstrating superiority over baseline methods, competitiveness with MO-ParamILS on individual scenarios, and an overall best performance.

Keywords

Automated algorithm configuration, multiobjective optimization, Bayesian optimization.

Correspondence to j.g.rook@utwente.nl.

Manuscript received: 28 June 2024; revised: 24 December 2024 and 3 March 2025; accepted: 3 March 2025.

1 Introduction

Algorithm parameters often have substantial influence on the performance achieved on a given task (see, e.g., Schede et al., 2022). However, exhaustive evaluation of all parameter settings is often infeasible, due to the large number of distinct configurations. Automated algorithm configuration (AAC) aims to automatically find well-performing parameter configurations for a set of tasks and substantially advance the state of the art for a broad range of AI problems, for example, SAT (Hutter et al., 2017), mixed integer programming (MIP) (Hutter et al., 2010), local search heuristics (Mascia et al., 2014), AI planning (Vallati et al., 2015), and machine learning (ML) (Thornton et al., 2013).[1] Although various AAC methods exist, only a few can efficiently handle mixed parameter spaces, that is, simultaneously optimize over continuous and categorical parameters, with conditional dependencies between them, whilst considering a set of (reasonably homogeneous) problem instances. Two such well-known general-purpose AAC methods are ParamILS (Hutter et al., 2009) and SMAC (Hutter et al., 2011; Lindauer et al., 2022).

Often algorithm performance is quantified using a single metric; for example, AAC can improve the running time of SAT solvers by up to three orders of magnitude (Hutter et al., 2017). However, this rarely reflects the often (conflicting) multiple objectives arising in practical situations (Rook et al., 2022), for example, running time and memory requirements. Also, there is a surging demand to achieve explainability and resource-efficiency in ML alongside low model error (Molnar et al., 2019; Benmeziane et al., 2021; Karl et al., 2023). Within evolutionary computation a plethora of (conflicting) measures exist to quantify the performance of an evolutionary optimizer, for example, the trade-off between convergence in objective and diversity in decision space (Preuß et al., 2024).

Thus, AAC methods providing a trade-off between multiple performance objectives are urgently needed. Without preferences on objectives, multiobjective (MO)-AAC methods find configurations as part of the a-priori unknown set of trade-off solutions. Unfortunately, few general-purpose MO-AAC methods exist. Previously, Blot et al. (2016) have extended the local-search-based ParamILS (Hutter et al., 2009) into an MO approach dubbed MO-ParamILS. However, considering the substantial performance improvements of SMAC over ParamILS (Hutter et al., 2011), it is reasonable to expect gains from an MO variant of SMAC.

Contributions. To this end, we propose the new multiobjective variant of SMAC, MO-SMAC, by adapting two key components of SMAC. Firstly, two options for sampling new configurations are evaluated: ParEGO (Knowles, 2006) together with any single-objective acquisition function (AF) and our suggested AF predicted hypervolume improvement (PHVI) maintaining a set of nondominated configurations. Secondly, our sophisticated intensification routine MO-Intensify efficiently evaluates configurations on different instances and determines which ones to finally return. In this paper, we first formulate the multiobjective algorithm configuration problem after which we discuss related work. Then we detail our approach for MO-SMAC, followed by an empirical evaluation of MO-SMAC against other configurators on six different scenarios, showing state-of-the-art performance of MO-SMAC. We conclude with a discussion and pointers for future work.

[1] In ML, algorithm parameters are called hyperparameters.

2 Background and Related Work

In this section, we introduce the algorithm configuration problem and relevant concepts from multiobjective optimization and multiobjective performance evaluation formally. We then cover related work from the literature.

2.1 Algorithm Configuration Problem

First, we formalize the single-objective AAC task. Let A be an algorithm with parameters chosen from a—generally multidimensional and mixed—configuration space Θ, and let Π be a set of problem instances. When algorithms have a stochastic component, the instance set Π can also be a Cartesian product of a set of random seeds and the actual problems instances. The goal of AAC is to find a configuration $\theta^* \in \Theta$ such that $c(\theta^*, \Pi) = \text{opt}_{\theta \in \Theta} c(\theta, \Pi)$; that is, θ^* optimizes (opt $\in \{\min, \max\}$) some quality or cost metric c on the instance set. Typically, the aggregation metric $c(\theta, \Pi)$ is obtained by running A with a fixed parameter configuration $\theta \in \Theta$ on all instances $I \in \Pi$, that is, by overloading c, $c(\theta, \Pi) = \frac{1}{|\Pi|} \cdot \sum_{\pi \in \Pi} c(\theta, \pi)$. Performance can be measured in different ways, depending on the optimization task. In classical combinatorial optimization scenarios, the runtime until A finds a (near-) optimal solution or the gap to a (known) optimal solution are typical choices. In ML, for example, classifier accuracy shall be maximized or its complexity shall be minimized for the sake of memory efficiency or interpretability.

The goal of multiobjective optimization (MOO) is to optimize multiple objective functions $c^i : \Theta \to \mathbb{R}$, $i \in \{1, \ldots, m\}$, $m \geq 2$ simultaneously. There is usually no single optimum in MOO, since objectives can be conflicting; for example, in our preceding example, a decrease in model complexity will usually lead to a decrease in model accuracy. Thus, in MOO, the concept of Pareto dominance (Ehrgott, 2005) is used: a configuration $\theta \in \Theta$ *dominates* another configuration $\theta' \in \Theta$, written as $\theta \prec \theta'$, if $c^i(\theta) \leq c^i(\theta') \, \forall i \in \{1, \ldots, m\}$, and there is at least one objective $j \in \{1, \ldots, m\}$ with $c^j(\theta) < c^j(\theta')$; w.l.o.g. we assume minimization of all objectives here. A configuration $\theta \in \Theta$ is *Pareto-optimal* if there is no other configuration in Θ that dominates θ. Given this notion, the goal in MOO, and thus for MO-AAC, is to find the set of optimal trade-off configurations $\Theta^* = \{\theta \in \Theta \mid \nexists \theta' \in \Theta : \theta' \prec \theta\}$ called the *Pareto set* and its image in the objective space, the *Pareto front* (PF). The AAC task is an optimization problem, and, consequently, evolutionary computation techniques are frequently used in AAC frameworks (Schede et al., 2022). Analogously, multiobjective evolutionary computation techniques can be used for addressing MO-AAC, which we also rely on for MO-SMAC. Additionally, we resort to commonly used MOO quality indicators in assessing the performance in our emperical evaluation in Section 4.

2.2 MOO Quality Indicators

Performance assessment of MOO-algorithms is non-trivial as several aspects matter, for example, the spread of the solutions in objective space and their closeness to the true (unknown) PF or an approximation thereof (i.e., MO performance measurement is also of multiobjective nature). Here, we use two well-known measures (see Audet et al., 2021, for a recent survey). The first one is the *hypervolume indicator* (HV) (Zitzler and Thiele, 1998). Given a point set $X \subset \mathbb{R}^m$ and an anti-optimal reference point $r \in \mathbb{R}^m$

the indicator is defined as a measure of the union of (hyper-)boxes

$$HV(X) = \lambda_m \left(\bigcup_{\substack{x \in X \\ x \prec r}} [x, r] \right),$$

where λ_m denotes the m-dimensional Lebesgue measure and $[x, r]$ is the (hyper-)box delimited by x from below and r from above. The hypervolume indicator—besides its appealing simplicity of interpretation—rewards both convergence to and coverage of the PF. Second, *inverted generational distance* (IGD+) (Ishibuchi et al., 2015) measures the average distance of points from a reference set to their closest neighbors in the approximation set while taking the dominance relation of the two into account. The latter makes this metric weakly Pareto compliant. These two indicators are used to validate and compare configurators in our experimental evaluation in Section 4.

In addition to the formerly mentioned indicators that measure the performance of a solution set, there are also measures for individual points in the context of the solution set. Crowding distance (CD) is a diversity measure used by the NSGA-II algorithm (Deb et al., 2002) as a secondary selection criterion that estimates the density of the points in objective space. For each point from the approximation set, it calculates the sum of the normalized distances between the objective values of its two nearest neighbors, calculated across all objectives; higher average CD values indicate a lower density and therefore a higher spread. MO-SMAC uses the crowding distance when configurations need to be removed from the solution set (see Section 3.2).

2.3 Related Work on AAC

Schede et al. (2022) provide an overview of single-objective (SO-)AAC in a recent survey, discussing challenges, perspectives, and existing configurators, such as irace (López-Ibáñez et al., 2016), ParamILS (Hutter et al., 2009), SMAC (Hutter et al., 2011), SPOT (Bartz-Beielstein et al., 2010), or GGA (Ansótegui et al., 2009). Furthermore, Bischl et al. (2023) explicitly focus on hyperparameter optimization (HPO) as a subclass of the general AAC framework and mainly applied to ML in the context of Auto-ML (Hutter et al., 2019). In contrast to general AAC, HPO usually works on a single dataset (e.g., a single instance in AAC) and focuses on solution quality metrics rather than algorithm runtimes (Eggensperger et al., 2018). HPO falls back to AAC by treating the different folds of a k-fold cross-validation as single instances (Thornton et al., 2013). Recently, Karl et al. (2023) provided a survey on multiobjective HPO. Despite the explicit focus on MO-HPO methods, they also identify the broader need for multiobjective algorithm configuration in other AI domains and also provide relevant scenarios and interesting (trade-off) objectives for ML scenarios. For example, obtaining models that are performant but also are efficient in their energy consumption. Additional objectives can also concern robustness, interpretability, or fairness. As urgently needed, several MO-extensions of SO-AAC were introduced. I/S-race (Miranda et al., 2015), S-race (Zhang et al., 2013), and SPRINT-Race (Zhang et al., 2015), fall into the category of racing algorithms of which several SO-AAC counterparts exist.

Most similar to our approach is MO-ParamILS (Blot et al., 2016), a multiobjective extension of ParamILS (Hutter et al., 2009) based on iterated local search. In its best-performing variant, it is combined with intensification, such that the number of instances for evaluating configurations is increased along with respective increasing quality, discarding dominated configurations at early stages. The biggest disadvantages

include the need for the discretization of the configuration space and its local search, making it unlikely to discover global optima in complex spaces. To the best of our knowledge, MO-ParamILS is considered the state-of-the-art for MO-AAC, and we will empirically compare against it as well as address its shortcomings using Bayesian optimization as a global optimization approach that can handle continuous parameters natively.

Moreover, Ugolotti et al. (2019) describe evolutionary multiobjective parameter tuning as a MO meta-optimization approach based on NSGA-II, which considers performance criteria of evolutionary algorithms, such as solution quality, convergence speed, and different fitness evaluation budgets. In contrast to our approach, these cannot handle the instance space efficiently and would require an evaluation of each configuration on all instances, which is inefficient as most configurations can be stopped early. The latter actually holds for most other MO optimization algorithms and MO-HPO frameworks that we are aware of and are therefore not considered in our empirical comparison in Section 4. Only in the context of noisy multiobjective optimization, alternative approaches for efficiently performing repeated evaluations exist, such as the rolling tide evolutionary algorithm (Fieldsend and Everson, 2015). Here, as long as solution points remain in the dominated set during search, more evaluations are performed on them. This algorithm is related to the intensification procedure in MO-SMAC as they have the same conceptual goals. Most other methods that deal with noise tend to have assumptions on the noise distribution (usually a Normal distribution). As we seek for the representiveness of the average performance of a (subset) of instances to present the actual average performance, such methods are considered unsuitable for MO-AAC.

Another work that follows a similar approach to our method was proposed by Koch et al. (2015), where sequential parameter optimization is extended with a multiobjective Bayesian optimization approach that can deal with noise. The effect of noise in evaluating a problem instance and mitigiating this by performing repeated evaluations is also investigated. They conclude that repeated evaluations are only desirable when there is much noise present. It remains unclear if these findings apply to AAC scenarios where the target domain is represented by a set of instances that usually have heteroscendentic noise, as Koch et al. focus on machine learning scenarios where a model is fitted on one dataset.

2.4 Related Work on Surrogate-Assisted MOO

When objective function evaluations are computationally expensive, sequential modelbased optimization (SMBO) is the method of choice. Such algorithms use surrogate modelling to approximate the true objective function with a cheap model. In a nutshell, the workflow is as follows: an initial set of points is sampled in the decision space, evaluated with the true objective, and an initial surrogate is built based on these observations. Following that, the algorithms perform the following steps iteratively until an evaluation budget is depleted: New points are proposed by an acquisition function (AF) that usually balances exploitation of the current model and exploration of uncertain areas of the model. The new points are evaluated and added to the set of points refining the model. Bayesian global optimization (BGO) (Močkus, 1975), commonly referred to as efficient global optimization (EGO), is a popular SMBO approach that uses a Gaussian process as surrogate model and uses the expected improvement as an acquisition function. One of its first multiobjective extensions of BGO was ParEGO (Knowles, 2006). ParEGO reduces the MO-problem to a single-objective version by weighted scalarization with the Tschebycheff augmented function. A surrogate model is then built on the

scalarized function. In order to cover the entire Pareto front ParEGO samples a random weight vector in each iteration. In contrast, *S*-metric selection EGO (SMS-EGO) (Ponweiser et al., 2008) uses an AF based on the HV contribution and does not rely on scalarization. Expected hypervolume improvement (EHVI) (Emmerich et al., 2006) is another multiobjective AF that shows good capability to guide the exploration process of SMBO algorithms, and efficient methods exist to compute it, for example with the KMAC algorithm (Yang et al., 2017). However, EHVI requires the variance of the surrogate models which in a multiobjective setting has a quadratic time complexity and is usually more computationally expensive to obtain than the actual acquisition function. Furthermore, there is an assumption that the surrogate models are initialized with a covering initial design of the search space. AAC scenarios usually have a limited number of evaluations, consequentially limiting the initial design, making the choice for EHVI a less obvious choice to use in MO-SMAC.

3 From SMAC to MO-SMAC

For our new MO-SMAC approach we use the default configuration as the initial design, which is common for AAC (Hutter et al., 2011). Furthermore, we use the random forest (RF) surrogate model, since it is known to work well in SO-AAC and supports the notion of problem instances (Hutter, Xu et al., 2014). The two key contributions made by MO-SMAC are a new intensification procedure, dubbed MO-Intensify, and two variants of handling objectives and generating configurations. Lastly, the SMBO interacts with the target function evaluator to evaluate the configurations on random seeds, budgets, and problem instances.

MO-SMAC is implemented in SMAC3 (Lindauer et al., 2022), which is a flexible open-source framework for AAC and HPO based on Bayesian optimization (BO) that extends preceding versions of SMAC (Hutter et al., 2011). For the sake of readability, we refer to SMAC3 as SMAC from here on. SMAC implements several BO approaches and provides presets (a.k.a. façades) for different use-cases, including black-box optimization, multi-fidelity optimization, and AAC. MO-SMAC is encoded in the new MO-AC façade. SMAC's native support for problem instances renders it a suitable basis for extension to general MO-AAC scenarios. In Figure 1, we illustrate the key components of SMAC and indicate how MO-SMAC (yellow) extends the framework. First, the scenario comprises the algorithm to optimize, the configuration space Θ, the optimization budget, and the problem instances $\pi \in \Pi$. The façades cover different use-cases and set up the different options for SMBO, including the initial design, the surrogate model, the AF, and the intensification procedure. The intensification procedure decides if a configuration should become part of the incumbent set, that is, the set of best configurations encountered so far. To become an incumbent in the MO setting, a configuration must be nondominated by all other previously evaluated configurations.

The SMBO loop is illustrated in Algorithm 1. We start with an empty set of incumbent configurations (Line 1). As long as the termination criterion is not met (e.g., walltime or number of function evaluations), we obtain a queue of challenger configurations aspiring to become part of the incumbent set with `getChallengers`; see Section 3.1. While there are still challengers in the queue and we do not decide to retrain the surrogate model (Line 4), we select one challenger to intensify against the incumbent set, which may lead to that challenger being added to the incumbent set (Line 7). Our intensification routine is detailed in Algorithm 2 and further explained in Section 3.2. The model is retrained once the challenger queue is empty or when the `needRetrain` procedure decides so. `needRetrain` evenly balances the walltime SMBO spends between

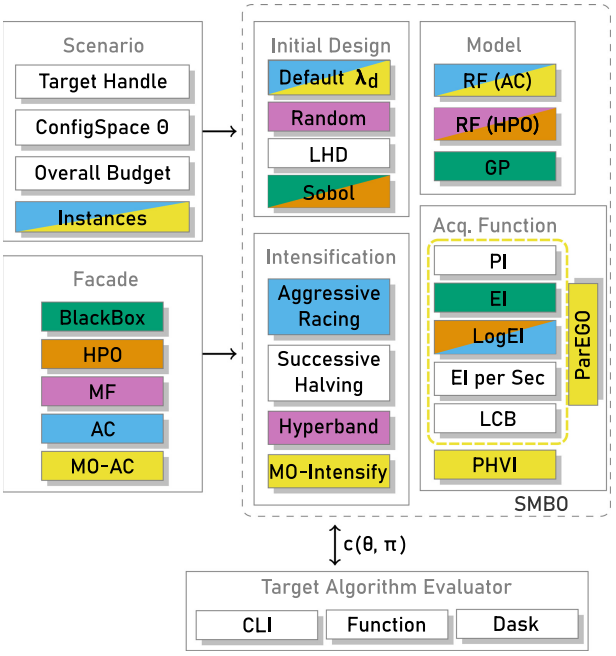

Figure 1: MO-SMAC (yellow). We extend SMAC3 by our intensification routine MO-Intensify and two configuration samplers: ParEGO together with any acquisition function or our predicted hypervolume improvement (PHVI). Figure adapted from Lindauer et al. (2022).

Algorithm 1: SMBO loop in SMAC

Input : configuration space Θ, instances Π, budget b

$\Theta_{inc} \leftarrow \{\}$ // Incumbent set
$\mathcal{R} \leftarrow \{\}$ // Runhistory
while termination criterion not met **do**
  $\Theta_{challengers} \leftarrow$ `getChallengers(`Θ`,` $\mathcal{R}$`)`;
  while $\Theta_{challengers} \neq \emptyset$ and not `needRetrain(`b`)` **do**
    $\theta_{new} \leftarrow$`pop(`$\Theta_{challengers}$`)`
    $\Theta_{inc}, \mathcal{R} \leftarrow$ `intensify(`θ_{new}, Θ_{inc}, Π, $\mathcal{R}$ `)`
return Θ_{inc}

running the challengers and finding new challengers when the budget B is walltime. For all budgets, it also ensures a minimum of two challenger intensifications before retraining, to allow sampled configurations to be intensified as well. In the following, we detail how MO-SMAC samples configurations, handles objectives, and our intensification approach.

3.1 Generating Configurations

In MO-SMAC, we offer two mechanisms for generating new configurations (`getChallengers`), namely ParEGO (Knowles, 2006) operating on a single surrogate model, and our newly added predicted hypervolume improvement (PHVI) as an acquisition function operating on one surrogate model per objective. Both mechanisms share the same procedure for generating the challenger queue. With all function evaluations

done by SMAC, a surrogate RF regression model is fitted, where the configuration and instance (features) serve as input vector and the label is the observed cost. MO-SMAC then uses that surrogate model to predict the performance of previously unseen configurations on the given instances. These predictions are used by the acquisition function to quantify the potential of this configuration. SMAC's `getChallengers` uses an efficient combination of global random search on the acquisition function followed by a local search to improve the configurations from the previous step further. Finally, `getChallengers` mixes in randomly sampled configurations with a probability of 0.5 at each position to ensure convergence to the optimal configuration in the limit (Hutter et al., 2011).

MO-SMAC-PHVI. One approach to handle multiple objectives is to maintain one surrogate model per objective. Predicted hypervolume improvement (PHVI), as a widely-used approach, computes the predicted improvement of adding a configuration to the PF. The hypervolume $\mathcal{H}$ is defined as the volume enclosed by the set of nondominated points and a reference point. This point needs to be dominated by all other points. The larger $\mathcal{H}$, the better the PF approximation is. Let C_{inc} be the matrix of the cost vectors $[\boldsymbol{c}_1, \ldots, \boldsymbol{c}_l]^T$ of all configurations in the incumbent set Θ_{inc} of size l. We can then calculate the contribution of the cost of a configuration to the PF approximation by

$$\Delta\mathcal{H}(\boldsymbol{c}_\theta, C_{inc}) = \mathcal{H}(C_{inc} \cup \{\boldsymbol{c}_\theta\}) - \mathcal{H}(C_{inc}).$$

Thus, PHVI can be defined as the scalar improvement on the HV based on the predicted costs as

$$\mathrm{PHVI}(\mu_\theta, \mathrm{M}_{inc}) = \Delta\mathcal{H}(\mu_\theta, \mathrm{M}_{inc}).$$

To account for the different sets of instances evaluated by each configuration so far, we utilize the surrogate model to predict the cost vector of the challenger μ_θ as well as the predicted cost matrix of the incumbents, $M_{inc} = [\mu_1, \mu_2, \ldots, \mu_l]^T$. In contrast, expected hypervolume improvement (EHVI) considers the variance of the predictions made by the surrogate model, making it more explorative but also computationally more expensive. The exploitative nature of PHVI is mitigated by interleaving random configurations, which is also the default for the AC façade in SMAC. Recently, empirical evidence (De Ath et al., 2021; De Winter et al., 2024) shows that using a purely exploitative AF can be beneficial for the performance, which strengthens our choice for using the PHVI. We normalize each objective by the current minimum and maximum observed costs in order to account for different objectives' scales and thus use 1.1 (i.e., a value closely beyond the worst achievable value) as reference point in each objective. PHVI together with our MO-Intensify routine is called MO-SMAC-PHVI.

MO-SMAC-PE. ParEGO aggregates the objective values into a single scalar by augmented Tchebycheff aggregation as

$$c = \max_{i=1,\ldots,m} (w_i \cdot c^i(\theta)) + \rho \cdot \sum_{i=1}^{m} w_i \cdot c^i(\theta),$$

where the aggregation weights $[w_1, \ldots, w_m]^T$ with $\sum_{i=1}^{m} w_i = 1$ are sampled uniformly at random and ρ is the balancing parameter of the approach, recommended to be set to 0.05 (Knowles, 2006). Therefore, only one surrogate model is required, which is trained with the scalarized objectives. By randomly varying the aggregation weights each time the surrogate model is trained, ParEGO implicitly aims to discover solutions spanning the Pareto front. Hence, the standard single-objective BO setup can be

Algorithm 2: MO-Intensify (`intensify`)

Input : Challenger θ_{new}, Incumbents Θ_{inc}, Instances Π, Runhistory $\mathcal{R}$

$\Pi_{eval} \leftarrow \bigcap_{i=0}^{|\Theta_{inc}|}$ `getTrials`(Θ^i_{inc})
$I \leftarrow \{\}$
$\boldsymbol{c}_{new} \leftarrow [(\infty)_{\times m}]$
while true **do**
  $\boldsymbol{c}_{old} \leftarrow \boldsymbol{c}_{new}$
  do
    // Evaluate challenger
    $I \leftarrow I \cup$ `sample`$(\Pi_{eval} - I)$
    $\boldsymbol{c}_{new}, \mathcal{R}_{new} \leftarrow$ `evaluate`(θ_{new}, I)
    $\mathcal{R} \leftarrow \mathcal{R} \cup \mathcal{R}_{new}$
  while $\boldsymbol{c}_{new} \not\prec \boldsymbol{c}_{old}$ and $\Pi_{eval} \neq I$
  // When the challenger ran on the incumbent instance subset
  if $I = \Pi_{eval}$ **then**
    // Recalculate Pareto front
    $\Theta_{inc} \leftarrow$ `getNonDominated`$(\Theta_{inc} \cup \{\theta_{new}\}, I)$
    if $|\Theta_{inc}| >$ max_incumbent_size **then**
      $\Theta_{inc} \leftarrow$`reduce`(Θ_{inc}, I)
      // Reduce on crowding distance
    break
  $\theta_{inc} \leftarrow$ `getClosestIncumbent`$(\theta_{new}, \Theta_{inc}, I)$
  $\boldsymbol{c}_{inc}, \mathcal{R}_{inc} \leftarrow$ `evaluate`(θ_{inc}, I)
  $\mathcal{R} \leftarrow \mathcal{R} \cup \mathcal{R}_{inc}$
  if $\boldsymbol{c}_{inc} \prec \boldsymbol{c}_{new}$, I **then**
    break // Reject the challenger
  // Add trial to the incumbents
$\theta_{inc} \leftarrow \arg\min_{\theta_{inc} \in \Theta_{inc}} |$`getTrials`$(\theta_{inc})|$
$\_, \mathcal{R}_{inc} \leftarrow$ `evaluate`$(\theta_{inc},$`pop`$(\Pi -$`getTrials`$(\theta_{inc})))$
return Θ_{inc}, $\mathcal{R} \cup \mathcal{R}_{inc}$

used. Here, we use expected improvement (EI) (Mockus et al., 1978) as the acquisition function to propose challenger configurations. ParEGO has the advantage of training only one surrogate model instead of one for each objective, which is computationally more efficient. A downside is that ParEGO is less able to handle concave-shaped Pareto fronts (Coello Coello, 1999). We dub ParEGO in combination with our intensification routine MO-Intensify MO-SMAC-PE.

3.2 Intensification

The generic outline of the intensification procedure (Algorithm 2) in (SO-)SMAC and MO-SMAC is as follows. Once a new challenger is obtained, its performance is compared against the incumbent set by iteratively evaluating the challenger on the same instances as the incumbents were evaluated. Recall that when the target algorithm has stochastic components an instance is actually a combination of a problem instance and a random seed. After each instance evaluation, we decide if the challenger should be compared against the incumbent. This intermediate comparison assesses if the challenger is sufficiently promising—based on the mean cost values of the instances the challenger ran on—to be further evaluated. Otherwise, the challenger is rejected, to prevent wasting computation on poorly performing configurations. Once the challenger has been evaluated on the same instances as the incumbents, it is decided whether it becomes part of the incumbent set. After acceptance or rejection, the incumbent configuration with the fewest evaluations is evaluated on a new instance. Thus, the

number of instances increases along with the quality of incumbent configurations. When the number of instances where all incumbent configurations were evaluated on increases, the underlying domination relations can change. When this occurs, the incumbent update procedure is triggered, which is explained below.

For MO-SMAC, we adjust the original intensification procedure of SMAC by modifying the comparison trigger, the intermediate comparison, and the incumbent set update. The resulting intensification procedure is described in Algorithm 2, and our modifications are the following:

When to compare challengers? We trigger a comparison between a challenger and the incumbent set when the new cost vector for a challenger dominates the old cost vector, denoted by $\boldsymbol{c}_{new} \prec \boldsymbol{c}_{old}$, where $\boldsymbol{c}_{old}$ is the cost at the time of the previous comparison. $\boldsymbol{c}_{new}$ is the average cost on the instances the configuration was evaluated on so far. After a comparison is triggered, the $\boldsymbol{c}_{new}$ becomes $\boldsymbol{c}_{old}$. The initial value of $\boldsymbol{c}_{old}$ is infinite for each element, forcing an intermediate comparison after the first evaluation. This domination-based scheme is also used in MO-ParamILS and it replaces the fixed, SMAC-default scheme of doubling the number of evaluations after each comparison. It can occur that $\boldsymbol{c}_{new}$ never dominated $\boldsymbol{c}_{old}$ after the first evaluation, resulting in the challenger being evaluated on all the instances the incumbents ran on.

How to compare challengers? For the comparison, we select the incumbent closest to the challenger in terms of Euclidean distance in the objective space. To equalize the objective dimensions, the cost vectors are first normalized, using min-max normalization by the minimum and maximum values of each objective observed since the start of the configuration run. When the closest incumbent dominates the challenger, the challenger is rejected.

There is a probability that we incorrectly reject a configuration that is actually good (type I error) or vice-versa (type II error), because the comparison is made on a subset of all the instances the incumbent set ran on. These probabilities likely increase when we compare the challenger against all incumbents. By comparing only against the closest incumbent, we reduce the possibility of making type I errors, which we consider more harmful than type II errors, at the cost of more type II errors.

How to update the incumbents? After the challenger and the incumbent configurations have been run on the same instances, we recompute the set of nondominated configurations based on the challenger and the incumbent set, keeping only nondominated configurations (`getNonDominated`). If the challenger belongs to the nondominated set, it is added to the incumbent set; otherwise, it is rejected. In addition, every incumbent that is dominated by any other incumbent configuration is removed from the incumbent set. In case the incumbent set holds more nondominated configurations than a user-specified maximum, then configurations with the lowest crowding distance (Deb et al., 2002) are removed using the `reduce` function (Line 14 in Algorithm 2). This ensures that the configurations in the incumbent set evenly cover the nondominated front even with few configurations such that the size of Π_{eval} keeps expanding at an allowable rate.

Algorithm 2 uses the following helper functions: `sample` returns one element from the provided set uniformly at random, `evaluate` returns the mean cost of each objective on the given set of instances plus the costs of each newly performed target algorithm run, and `getTrials` returns all the instances on which a given configuration has been evaluated. When a configuration θ did not have any evaluation on an instance in the provided instance set I, then one evaluation is carried out before the `evaluation` function returns the cost vector $\boldsymbol{c}$.

Table 1: MO-AAC scenarios used in our experiments. For the parameter types, R refer to a real, I to integer, and C to categorical parameter, respectively. For the Instances, the first number represents the size of the training set and the following number the test set.

Domain	Solver	Instance set	Objectives	Budget	Parameters	Instances
EMOA	Omni-optimizer	Multi-Modal	HV + SP	500 runs	5R+3C	24+8
MIP	CPLEX	Regions200	Gap + cutoff Gap + CPU time	24h	4R+7I+64C	1000+1000
SAT	CMS	QUEENS	CPU time + memory	6h	3R+15I+11C	352+352
ML	RF	Students	Precision + recall *previous* + size Accuracy + size	500 runs	4R+6I+9C	10+1

One key aspect of the multiobjective setting is that we are no longer dealing with a single incumbent, but rather have multiple configurations that ideally approach and cover the Pareto front across the objectives. The maximum number of configurations in the incumbent set introduces a new parameter for MO-SMAC; it represents the trade-off between the quality and coverage of the front and is maintained by the `reduce` function.

4 Empirical Evaluation

We empirically compare our two MO-SMAC variants, MO-SMAC-PHVI and MO-SMAC-PE, on seven scenarios that originate from three different AI domains, described in Table 1, against MO-ParamILS and three baseline configurators. Our experiments were conducted on a HPC cluster with 2 Intel® Xeon® E5-2683 v4 CPUs running at 2.1 GHz and 94 GB of memory per node. The implementations and experimental scripts can be found at https://github.com/jeroenrook/MOSMAC-ECJ. The implementation will also become part of the SMAC package. The empirical evaluation follows the best-practices for running algorithm configurators (Eggensperger et al., 2019) and used the protocols outlined in Van Der Blom et al. (2022), ensuring that our results reflect practical performance. Furthermore, we decided to use the baselines and scenarios that were used in the MO-ParamILS study (Blot et al., 2016).

4.1 Experimental Setup

The MO-SMAC variants use the components as described in Section 3. Specifically, we set the maximum number of incumbents to 10, use crowding distance in case the incumbent set needs to be reduced, run at most 2,000 instances per configuration, and sample 5,000 configurations for the global random search on the acquisition function. These parameters are based on the defaults used in the AC-façade of SMAC (Lindauer et al., 2022).

MO-AAC Scenarios. The MIP and SAT scenarios originate from ACLib (Hutter, López-Ibáñez et al., 2014) and are extended to multiple MO-AAC scenarios by focusing on different sets of objectives. These extensions will be integrated into ACLib or any successor thereof to enable others to easily make use of these. MIP objectives are to minimize the *gap* between the solution found when the MIP solver is halted and the known optimal solution. The second objective is either running time *cutoff* or the *running time* of the MIP solver. The difference between the two is that the cutoff is essentially a direct mapping of the cutoff parameter and the running time is the average running time. For both scenario variants the cuttoff parameter is included in the configuration

space. The MIP solver CPLEX is used in combination with the Regions200 benchmark set, which consists of a training and testing set of 1,000 different problem instances each. The configuration space of CPLEX is extended with a cutoff parameter that can take any $[1, 10]_{\mathbb{Z}}$ seconds. This allows to meaningfully compare the SO baseline configurator.

The SAT scenario involves minimization of average *running time*, using the PAR10-score (Hutter et al., 2009) as well as mean *memory* usage and has a cutoff time of 60s. Here, the SAT solver is CryptoMiniSAT (CMS) and the QUEENS benchmark comprises a training and testing set of 352 problem instances each. The ML scenario configures a random forest classifier for three different objective combinations. Model *size* is measured by the total number of nodes of all the trees in the resulting model.

In this ML scenario, the Students (Realinho et al., 2021) dataset is split into a training and testing partition using a 80%/20% split. The training set consists of 10 instances, each representing one fold of a 10-fold split on the training partition. The testing set comprises one instance where the random forest is fitted with the full training partition and reports the performance metrics *precision* and *recall* on the test partition of the dataset.

The EMOA scenario originates from Rook et al. (2022) where the multiobjective optimizer Omni-optimizer is configured to maximize convergence towards the Pareto front and to have maximum diversity in the decision space. These objectives are measured by the hypervolume and the Solow-Polasky indicators, respectively. The instance set consists out of 33 multi-model multiobjective optimization problems. Twenty-four are used for training and eight for testing. Omni-optimizer is given an evaluation budget of 20,000.

All algorithms have a configuration space with different parameter types as well as conditional parameter dependencies. A respective summary can be found in table 1. The Regions200 (MIP scenarios) and QUEENS (SAT scenario) benchmarks come with instance features that are used by the surrogate model.

Baselines. Our MO-SMAC variants are compared against MO-ParamILS and three baseline methods; the default configuration, SO-SMAC, and random search. For MO-ParamILS, the parameter spaces were discretized to 25 values for each continuous or integer parameter, and we ran the MO-FocusedILS variant with the settings described by Blot et al. (2016). For SO-SMAC, we divided the configuration budget evenly over each objective and ran SMAC to find the best configuration for each individual objective, using the same settings described by Hutter et al. (2011). In the MIP scenarios, we divided the budget for SMAC proportionally to each cutoff value $\{1, 2, 3, 5, 10\}$, resulting in five configuration scenarios for SMAC. In the ML scenarios, where the objectives include precision and recall, SMAC was run with the F_1-score as additional objective. For random search, we get new challengers by random sampling from the configuration space and are evaluated on a fixed random sample of 25 problem instances.

Evaluation Scheme. Due to the stochastic nature of the configurators, we ran each of them 20 times with different random seeds on each scenario in the training sets. This resulted in 20 incumbent sets. To study the practical capabilities of the MO-configurators, we sub-sampled n (independent) runs from our 20 repeated runs and determined an overall approximated Pareto front from all nondominated incumbent configuration sets in these n runs on a common base of at most 100 instances sampled without replacement from the training set. To get robust statistics, we repeated this sampling procedure $\binom{20}{2} = 190$ times, except for $n \in \{1, 19, 20\}$ where we enumerated all possible combinations of runs. We settled on 190 samples per run size as this is the least arbitrary choice that provides an equal number of samples for the majority of run

sizes ($2 \leq n \leq 18$). Subsequently, we measured the performance of these nondominated configurations on the test set, which holds disjoint instances from the training set. This protocol simulates multiple (parallel) independent runs to account for stochasticity and then the best performing configuration is selected as advised (Eggensperger et al., 2018). This is different from assessing the average performance and variance across single runs, as it more accurately reflects real-world usage scenarios. It is important to note that n runs effectively multiply the scenario budget with a factor of n.

In the MO setting, performing more than one run yields an interesting dynamic between the number of runs and the resulting quality scores. For example, two runs that find configurations in separate parts of the PF can each have a low HV, but in combination can yield strong improvements on the HV since the configurations now cover the PF better.

Performance of configurators was assessed using HV and IGD+ described in Section 2. In order to make the results comparable across scenarios, we use min-max normalization of objectives over all observed performances on the test set. The reference point, required to compute the HV, was therefore set to 1.1 for each objective. We also composed a reference set, required for the IGD+ indicator, by creating a nondominated set of configurations from all runs on a scenario, which is common practice for MOO problems with unknown Pareto fronts (Audet et al., 2021; Li and Yao, 2020).

To obtain the trajectory over multiple configurator runs, the trajectories from individual runs were combined. When at a certain time step any of the individual trajectories change their incumbent, all incumbents at that moment were stored in the combined trajectory along with the time step. Each entry in the resulting combined trajectory was filtered by only keeping the nondominated configurations based on their performance on the instances from the training set. The same sampling procedure of obtaining 190 subsets of size 8 from the 20 repeated runs was performed, resulting in 190 combined trajectories for each configurator and scenario pair. Since our (SO-)SMAC baseline internally has a configuration run for each objective with an adjusted budget, the trajectories are combined by reverting these budget adjustments. For example, when there are two objectives, the time steps of the resulting two internal trajectories that ran with half the budget are doubled. The trajectories for MO-ParamILS can only be reconstructed with the walltime and not the number of target algorithm runs. Therefore, the trajectories are represented only with walltime, causing—only in the ML and EA scenarios—that the trajectories can terminate at varying moments. The trajectories remain comparable this way, but perceivably favor MO-ParamILS and Random Search since they do not have the overhead of finding new configurations with BO. The alternative would have been to map the number of target algorithm runs by the total running time of an MO-ParamILS run, which is likely to end in a unrealistic representation of the trajectory, since the runtime on each instance can substantially differ.

4.2 Results

The aggregated results on the test sets with the nondominated configurations of each configurator of all sampled combinations of size 8 and 16 are presented in Table 2. We choose 8 and 16, as these are often the number of CPU cores found in modern computers and thus limits the number of parallel runs. The convergence of the mean quality performance with varying combination sizes (see Figures 5 and 6) strengthens the belief that different sizes will not show drastically different results.

Table 2 reveals that MO-SMAC-PHVI, on average, produces quantitatively better approximated PFs compared to the other methods and ranks best for both indicators. Especially for the IGD+ indicator, this configurator shows strong performance,

Table 2: Mean performance of the HV and IGD+ indicators of the 190 randomly sampled combinations of size 8 and 16 from the set of 20 independent runs. The performance ranking of the configurators is put in brackets. Best performing results and statistically tied results with the best are made boldface. A two-sided Mann-Whitney-U test was performed on all pairwise combinations between the best and the other configurators. A Holm-Bonferroni multiple test correction was performed with $\alpha = 0.05$.

	MO-SMAC-PHVI	MO-SMAC-PE	MO-ParamILS	Random Search	SMAC	Default
			Hypervolume, Runs = 8			
EMOA-(hv, sp)	1.045 (2)	**1.066 (1)**	1.004 (4)	1.031 (3)	0.818 (5)	0.148 (6)
MIP-(gap, cutoff)	0.986 (3)	0.923 (5)	**1.088 (1)**	0.922 (6)	1.031 (2)	0.957 (4)
MIP-(gap, runtime)	1.196 (3)	1.190 (4)	**1.200 (1)**	1.185 (6)	1.197 (2)	1.188 (5)
ML-(accuracy, size)	**1.187 (1)**	1.167 (3)	1.182 (2)	1.078 (5)	1.130 (4)	0.973 (6)
ML-(precision, recall)	**0.886 (2)**	**0.888 (1)**	0.858 (3)	0.770 (5)	0.819 (4)	0.770 (6)
ML-(precision, recall, size)	**1.160 (2)**	**1.160 (1)**	1.149 (3)	1.110 (5)	1.140 (4)	0.862 (6)
SAT-(runtime, memory)	**1.048 (1)**	**1.025 (3)**	0.968 (4)	0.907 (5)	1.040 (2)	0.606 (6)
Average ranking	**2.0**	2.6	2.6	5.0	3.3	5.6
			IGD+, Runs = 8			
EMOA-(hv, sp)	0.058 (2)	**0.036 (1)**	0.075 (4)	0.060 (3)	0.218 (5)	0.720 (6)
MIP-(gap, cutoff)	0.102 (3)	0.147 (6)	**0.012 (1)**	0.146 (5)	0.060 (2)	0.121 (4)
MIP-(gap, runtime)	0.011 (3)	0.018 (5)	**0.004 (1)**	0.022 (6)	0.006 (2)	0.016 (4)
ML-(accuracy, size)	**0.002 (1)**	0.004 (3)	0.004 (2)	0.011 (4)	0.016 (5)	0.194 (6)
ML-(precision, recall)	**0.033 (1)**	0.048 (2)	0.048 (3)	0.098 (5)	0.088 (4)	0.103 (6)
ML-(precision, recall, size)	**0.007 (1)**	0.014 (3)	0.008 (2)	0.018 (5)	0.016 (4)	0.186 (6)
SAT-(runtime, memory)	**0.077 (2)**	**0.076 (1)**	0.134 (4)	0.185 (5)	**0.081 (3)**	0.369 (6)
Average ranking	**1.9**	3.0	2.4	4.7	3.6	5.4

Table 2: Continued.

	MO-SMAC-PHVI	MO-SMAC-PE	MO-ParamILS	Random Search	SMAC	Default
			Hypervolume, Runs = 16			
EMOA-(hv, sp)	1.060 (2)	**1.078 (1)**	1.030 (4)	1.043 (3)	0.872 (5)	0.148 (6)
MIP-(gap, cutoff)	1.006 (3)	0.953 (6)	**1.097 (1)**	0.953 (5)	1.041 (2)	0.957 (4)
MIP-(gap, runtime)	1.198 (2)	1.193 (4)	**1.202 (1)**	1.189 (5)	1.197 (3)	1.188 (6)
ML-(accuracy, size)	**1.198 (1)**	1.171 (3)	1.182 (2)	1.109 (5)	1.168 (4)	0.979 (6)
ML-(precision, recall)	**0.913 (1)**	**0.911 (2)**	0.894 (3)	0.797 (5)	0.826 (4)	0.780 (6)
ML-(precision, recall, size)	1.170 (2)	**1.176 (1)**	1.158 (3)	1.117 (5)	1.150 (4)	0.870 (6)
SAT-(runtime, memory)	**1.154 (1)**	1.077 (4)	1.078 (3)	0.974 (5)	1.140 (2)	0.607 (6)
Average ranking	**1.7**	3.0	2.4	4.7	3.4	5.7
			IGD+, Runs = 16			
EMOA-(hv, sp)	0.050 (2)	**0.028 (1)**	0.059 (4)	0.055 (3)	0.180 (5)	0.720 (6)
MIP-(gap, cutoff)	0.087 (3)	0.126 (6)	**0.003 (1)**	0.121 (5)	0.053 (2)	0.121 (4)
MIP-(gap, runtime)	0.009 (3)	0.015 (4)	**0.001 (1)**	0.020 (6)	0.004 (2)	0.016 (5)
ML-(accuracy, size)	**0.001 (1)**	0.003 (2)	0.004 (3)	0.008 (4)	0.009 (5)	0.194 (6)
ML-(precision, recall)	**0.020 (1)**	0.035 (3)	0.028 (2)	0.085 (5)	0.073 (4)	0.098 (6)
ML-(precision, recall, size)	**0.005 (1)**	0.009 (3)	0.007 (2)	0.014 (5)	0.012 (4)	0.186 (6)
SAT-(runtime, memory)	**0.022 (1)**	0.044 (3)	0.051 (4)	0.118 (5)	0.025 (2)	0.368 (6)
Average ranking	**1.7**	3.1	2.4	4.7	3.4	5.6

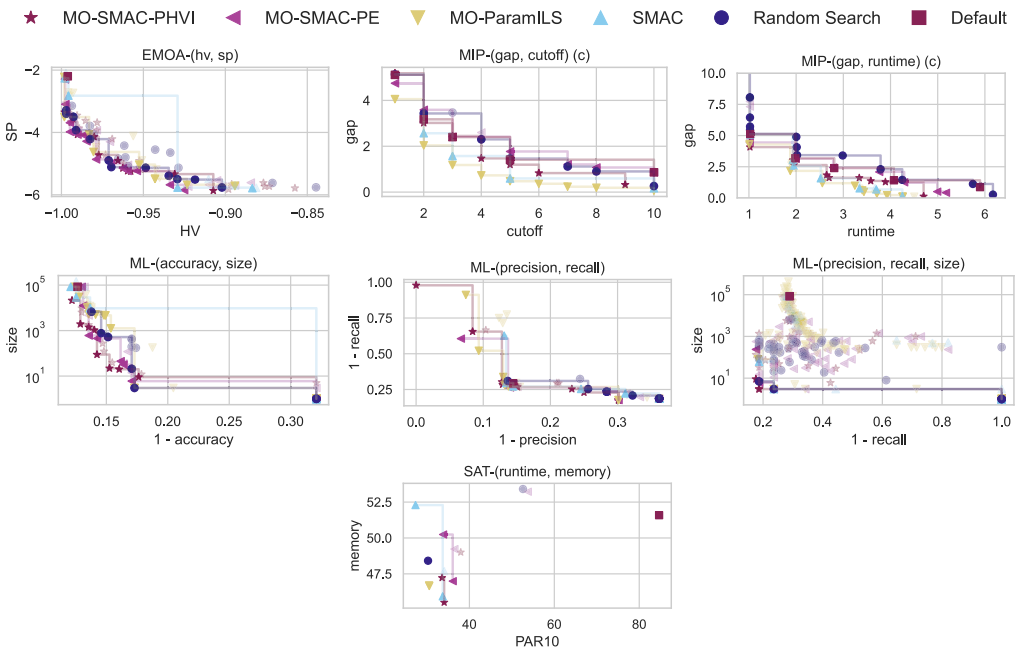

Figure 2: PF approximations of the configurators on all six different scenarios; these are the nondominated points from the union of the incumbent sets of the first eight runs (test set). For the ML scenario with three objectives, only the first two objectives are shown.

indicating that the configurations in the incumbent set are close to the approximated PF. MO-SMAC-PE produces comparable results to MO-SMAC-PHVI and MO-ParamILS, but has only a tied second rank for HV with eight runs and ranks third otherwise; we note that this is mainly due to the bad performance for the MIP scenarios. In this scenario, MO-ParamILS significantly outperforms all other methods for both quality indicators. This is also clearly reflected by the incumbent set fronts of the MIP scenario shown in Figure 2. Unsurprisingly, random search and the default often rank low, where the default configuration has the lowest rankings. Since the indicators of the default are computed with only one configuration, this is to be expected. The SMAC baseline, however, is surprisingly competitive in MIP and SAT scenarios, yielding nondominated sets with high HV-values. The weaker performance on the IGD+ indicator for SMAC suggests that in these two scenarios the approximated PFs are covered, but that it is unable to find configurations close to the front. The differences between 8 and 16 runs are limited to small shifts in the underlying rankings. As expected, the absolute performances are higher when more runs are considered. Overall, MO-SMAC-PHVI benefits the most from performing more runs in comparison to the other configurators based on the average rankings.

Fronts of the incumbent set. The fronts in Figure 2 show that in our ML scenarios, both MO-SMAC variants are able to find a set of configurations that are evenly distributed across the PF. MO-ParamILS, on the other hand, finds configurations that are distant from the approximated PF in the ML-(accuracy,size) scenario, but has the ability to find configurations on the extremes on the front. The latter explains the strong

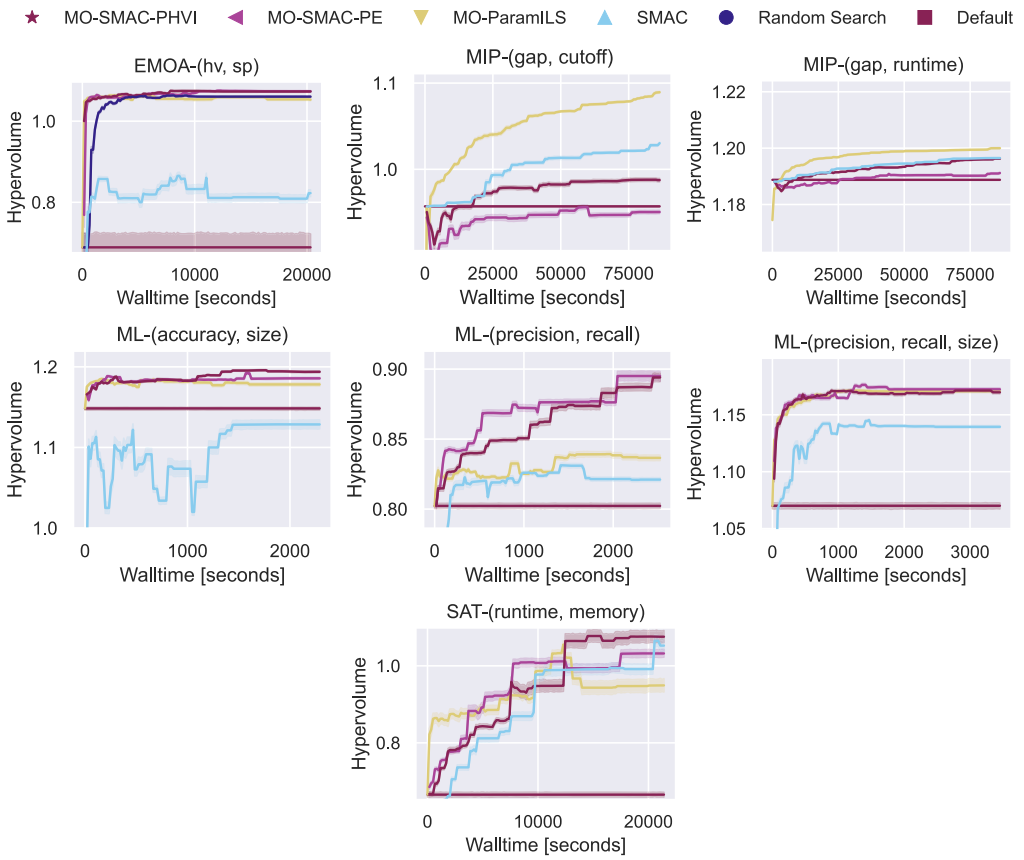

Figure 3: Hypervolume trajectory plots. Each line represents the mean performance over the underlying 190 sampled trajectories where each trajectory is compiled out of 8 configuration runs out of 20. The hypervolume is obtained of the incumbents' performances on the test set.

performance in terms of HV. Another interesting observation is that in the ML-(precision, recall) scenario (Figure 2), MO-SMAC-PE, and MO-ParamILS were able to find single configurations that yielded higher F_1-scores compared to the configurations found with SMAC configuring for the F_1-score. An additional one-sided Wilcoxon signed-rank test (with a Holm-Bonferroni multiple test correction) on the configurations with the highest F_1-score of each configuration run confirms this observation. This suggests that by using established performance indicators that aggregate underlying objectives, such as the F_1-score, sub-optimal models are found, while an MO approach can produce better models. The SAT scenario fronts indicate that the trade-off between running time and memory usage is limited. This explains the relative strong performance of the SMAC baseline in this scenario.

Trajectories. The trajectories, displayed in Figures 3 and 4, all follow a similar convergence trend and seem to stabilize near the end with regard to their underlying configurator rankings and coincide with the results from Table 2. The plots also indicate that the given configuration budgets were sufficient. In the ML scenarios with model size as an objective, (SO-)SMAC underperforms as this objective likely conflicts with the other

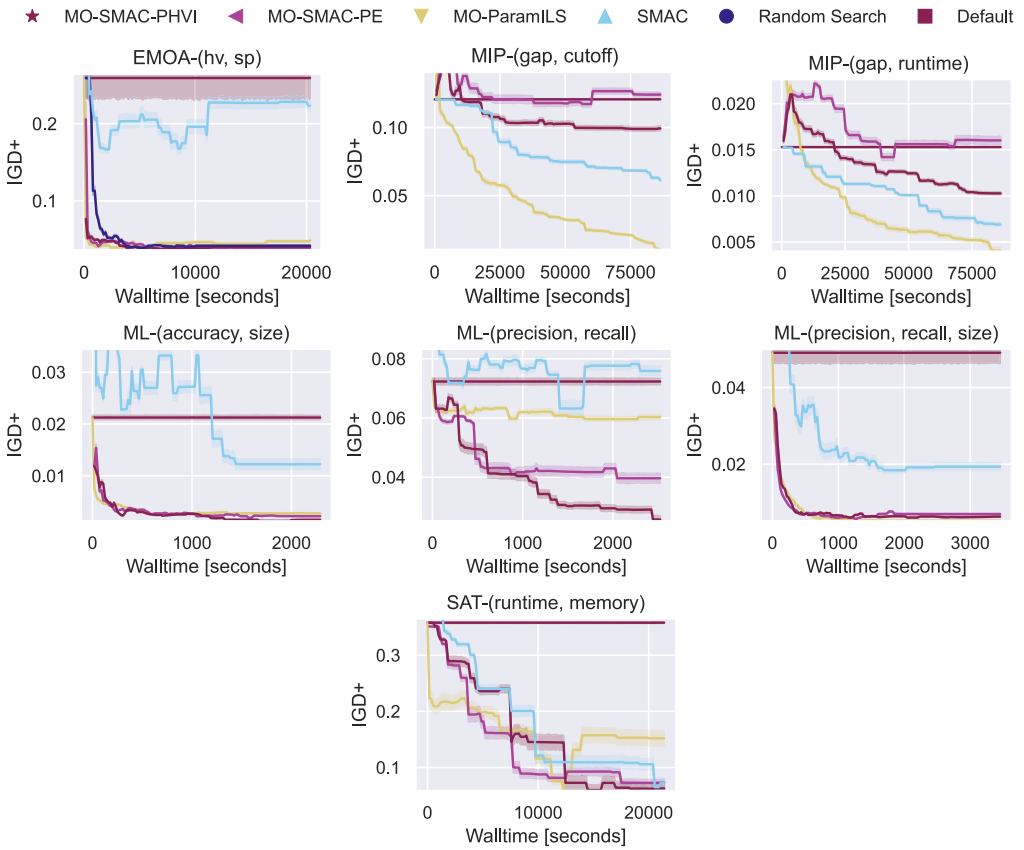

Figure 4: IGD+ trajectory plots. Each line represents the mean performance over the underlying 190 sampled trajectories where each trajectory is compiled out of 8 configuration runs out of 20. The IGD+ score is obtained from the incumbents' performances on the test set.

objectives and does not find solutions on the trade-off. In the SAT scenario, MO-ParamILS converges more quickly in the beginning compared to the MO-SMAC variants and Random Search, but also plateaus earlier after which it is overtaken by MO-SMAC variants and SMAC. This early dominant performance of MO-ParamILS might be caused by the random initialization of each run of MO-ParamILS, where for the others all runs start with the the default configuration. The most surprising trajectories are found in the MIP scenarios. Here, the performance of both MO-SMAC variants and the Random Search baseline get worse at the beginning of the configuration runs, where only MO-SMAC-PHVI seems to *catch on* halfway through the search and eventually starts having a solution set that is better than the default configuration. A hypothesis for this observation is that the large search space of CPLEX is challenging for the surrogate models when the number of samples is low, especially in combination with the cutoff objective, which is direct mapping from the cutoff parameter in the configuration space. MO-ParamILS is not affected by this as it does a direct local search and the SMAC baseline and the default configuration neither as they have predefined spread over the cutoff/runtime objective already.

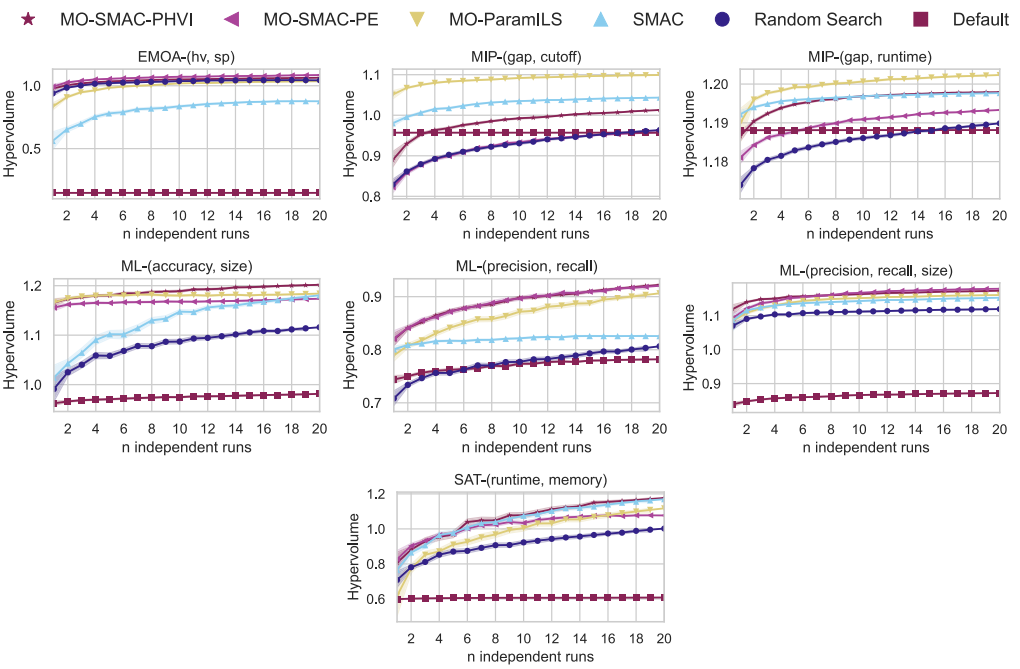

Figure 5: The mean hypervolume of each configurator set out over the differently sized combinations of individual runs. For each size n, we sampled 190 combinations from all possible combinations. For sizes $n \in \{1, 19, 20\}$ we used all combinations.

Number of independent runs. Figures 5 and 6 show the mean performance of the configurators with varying sizes of the combinations of runs for two exemplary and representative scenarios. There, we see the expected trend that performing more independent runs results in higher quality incumbent sets. More interestingly, we see that MO-SMAC-PHVI, MO-SMAC-PE, and MO-ParamILS exceed the baseline methods in performance, but between them achieve more similar performance as more runs are conducted. The two MO-SMAC variants stand out compared to MO-ParamILS by showing faster convergence with a small number of runs.

Key insight. The overall impression and conclusion from the results are that both MO-SMAC-PHVI and MO-SMAC-PE are competitive in their performance in comparison to MO-ParamILS. MO-SMAC-PHVI showed the most consistent good performance over all six scenarios. Another advantage of the MO-SMAC variants is that they also show strong performance with a small set of independent runs, which suggests that MO-SMAC is satisfyingly robust.

5 Discussion and Conclusions

In this paper, we presented our generic multiobjective automated algorithm configurator MO-SMAC, which is an extension of the single-objective SMAC procedure. SMAC is modified to find a set of configurations that lie on the trade-off between two or more objectives instead of finding only a single configuration. To achieve this, we provided two complementary approaches for the acquisition function and surrogate model to handle multiple performance objectives. The first approach, MO-SMAC-PE, uses ParEGO, which scalarizes the objectives such that the surrogate model and acquisition

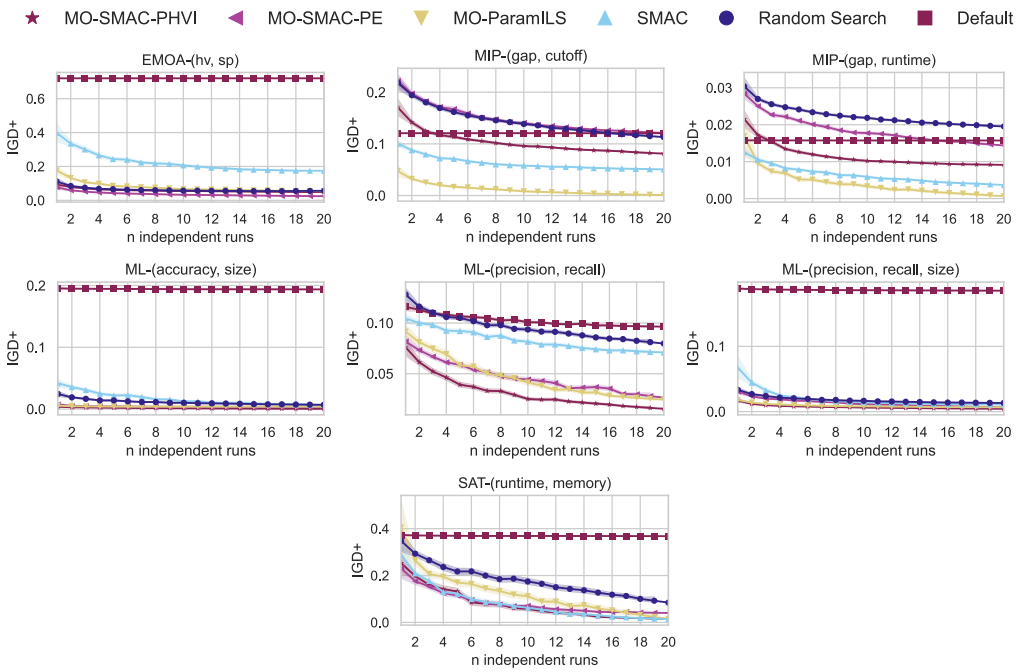

Figure 6: The mean IGD+ scores of each configurator set out over the differently sized combinations of individual runs. For each size n, we sampled 190 combinations from all possible combinations. For sizes $n \in \{1, 19, 20\}$ we used all combinations.

function operate in a single-objective manner. The second approach, MO-SMAC-PHVI, uses a new and sophisticated acquisition function, predicted hypervolume improvement (PHVI), that for each objective has its own surrogate model and predicts the HV improvement of a new configuration on the incumbent set of configurations. Furthermore, we introduced MO-Intensify, our intensification routine for the multiobjective setting, which decides on when and how to compare challengers with the incumbent set prematurely and how the incumbent set is updated. All these three components of MO-Intensify rely on Pareto dominance.

In an empirical evaluation of MO-SMAC, we compared the two variants, MO-SMAC-PHVI and MO-SMAC-PE, against MO-ParamILS and three baseline methods on six different scenarios originating from three AI domains: MIP, ML, and SAT. The results revealed that the resulting incumbent sets of MO-SMAC-PHVI over multiple runs are close to the approximated Pareto fronts and also completely cover these fronts uniformly. Individual runs of the configurators showed that MO-SMAC-PHVI is able to find strongly performing incumbent sets, albeit not always significantly better compared to the other MO-AAC configurators. MO-SMAC-PHVI also obtains the best average ranks across all scenarios, demonstrating robustness.

Besides our quantitative evaluation, MO-SMAC has several qualitative advantages over MO-ParamILS. The SMAC framework holds various alternatives to components of the configurator, such as different surrogate models and initial sample procedures. Furthermore, components of SMAC can be customized by users themselves. This allows MO-SMAC to be more versatile and adjustable to specific needs. Another advantage is that the SMAC package is actively maintained by a large community. Therefore, in

combination with the observed performances, we conclude that MO-SMAC-PHVI is preferred over the other configurators.

In future work it would be interesting to see how MO-SMAC performs and competes in other application areas, such as in multiobjective (continuous) optimization and other HPO areas like neural architecture search (NAS). In addition, we see several other directions for future work on MO-SMAC. A straightforward extension would be to complement MO-SMAC with other prominent capabilities of SMAC, such as multi-fidelity optimization. Another promising direction would be to not only asses configurations based on their performance, but also regarding their diversity in parameter values. This succeeds however, the need for a better understanding of MO configuration landscapes. In turn, this could facilitate the automatic tuning and selection of configurators themselves. Additionally, investigating the influence of parameters on the trade-offs between the objectives might lead to more efficient MO-AAC procedures. At last, in case only one configuration can finally be chosen, we plan to combine MO-SMAC with preferential learning using human-in-the-loop approaches, which is very much in line with the concept underlying, for example, preference-based multiobjective evolutionary optimization approaches (Branke et al., 2008).

Acknowledgments

The authors would like to thank Christian Grimme for his valuable input during the initial stages of this project. Carolin Benjamins acknowledges funding by the German Research Foundation (DFG) under LI 2801/7-1. Marius Lindauer also acknowledges support by the German Federal Ministry of the Environment, Nature Conservation, Nuclear Safety, and Consumer Protection (GreenAutoML4FAS project no. 67KI32007A).

References

Ansótegui, C., Sellmann, M., and Tierney, K. (2009). A gender-based genetic algorithm for the automatic configuration of algorithms. In *Principles and practice of constraint programming*, pp. 142–157. Springer Berlin Heidelberg.

Audet, C., Bigeon, J., Cartier, D., Le Digabel, S., and Salomon, L. (2021). Performance indicators in multiobjective optimization. *European Journal of Operational Research*, 292(2):397–422. 10.1016/j.ejor.2020.11.016

Bartz-Beielstein, T., Lasarczyk, C., and Preuss, M. (2010). The sequential parameter optimization toolbox. In *Experimental methods for the analysis of optimization algorithms*, pp. 337–362. Springer Berlin Heidelberg.

Benmeziane, H., Maghraoui, K. E., Ouarnoughi, H., Niar, S., Wistuba, M., and Wang, N. (2021). Hardware-aware neural architecture search: Survey and taxonomy. In Zhou, Z., editor, *International Joint Conferences on Artificial Intelligence 2021*, pp. 4322–4329.

Bischl, B., Binder, M., Lang, M., Pielok, T., Richter, J., Coors, S., Thomas, J., Ullmann, T., Becker, M., Boulesteix, A., Deng, D., and Lindauer, M. (2023). Hyperparameter optimization: Foundations, algorithms, best practices, and open challenges. *WIREs Data Mining and Knowledge Discovery*, 13(2):e1484. 10.1002/widm.1484

Blot, A., Hoos, H., Jourdan, L., Kessaci-Marmion, M., and Trautmann, H. (2016). MO-ParamILS: A multi-objective automatic algorithm configuration framework. In *Learning and Intelligent Optimization*, volume 10079, pp. 32–47. Springer International Publishing. 10.1007/978-3-319-50349-3_3

Branke, J., Deb, K., Miettinen, K., and Słowiński, R., editors (2008). *Multiobjective optimization: Interactive and evolutionary approaches*. Springer.

Coello Coello, C. (1999). A comprehensive survey of evolutionary-based multiobjective optimization techniques. *Knowledge and Information Systems*, 1(3):269–308. 10.1007/BF03325101

De Ath, G., Everson, R. M., Rahat, A. A. M., and Fieldsend, J. E. (2021). Greed is good: Exploration and exploitation trade-offs in Bayesian optimisation. *ACM Transactions on Evolutionary Learning and Optimization*, 1(1):1–22. 10.1145/3425501

De Winter, R., Milatz, B., Blank, J., Van Stein, N., Bäck, T., and Deb, K. (2024). Parallel multiobjective optimization for expensive and inexpensive objectives and constraints. *Swarm and Evolutionary Computation*, 86:101508. 10.1016/j.swevo.2024.101508

Deb, K., Pratap, A., Agarwal, S., and Meyarivan, T. (2002). A fast and elitist multiobjective genetic algorithm: Nsga-ii. *IEEE Transactions on Evolutionary Computation*, 6(2):182–197. 10.1109/4235.996017

Eggensperger, K., Lindauer, M., Hoos, H., Hutter, F., and Leyton-Brown, K. (2018). Efficient benchmarking of algorithm configurators via model-based surrogates. *Machine Learning*, 107(1):15–41. 10.1007/s10994-017-5683-z

Eggensperger, K., Lindauer, M., and Hutter, F. (2019). Pitfalls and best practices in algorithm configuration. *Journal of Artificial Intelligence Research*, 64:861–893. 10.1613/jair.1.11420

Ehrgott, M. (2005). *Multicriteria optimization*. Springer-Verlag.

Emmerich, M. T., Giannakoglou, K. C., and Naujoks, B. (2006). Single- and multiobjective evolutionary optimization assisted by Gaussian random field metamodels. *IEEE Transactions on Evolutionary Computation*, 10(4):421–439. 10.1109/TEVC.2005.859463

Fieldsend, J. E., and Everson, R. M. (2015). The rolling tide evolutionary algorithm: A multiobjective optimizer for noisy optimization problems. *IEEE Transactions on Evolutionary Computation*, 19(1):103–117. 10.1109/TEVC.2014.2304415

Hutter, F., Hoos, H., and Leyton-Brown, K. (2010). Automated configuration of mixed integer programming solvers. In *Proceedings of the 7th International Conference on Integration of AI and OR Techniques in Constraint Programming for Combinatorial Optimization Problems*, volume 6140, pp. 186–202.

Hutter, F., Hoos, H., and Leyton-Brown, K. (2011). Sequential model-based optimization for general algorithm configuration. In *Learning and Intelligent Optimization*, volume 6683, pp. 507–523. Springer Berlin Heidelberg. 10.1007/978-3-642-25566-3_40

Hutter, F., Hoos, H., Leyton-Brown, K., and Stützle, T. (2009). ParamILS: An automatic algorithm configuration framework. *Journal of Artificial Intelligence Research*, 36:267–306. 10.1613/jair.2861

Hutter, F., Kotthoff, L., and Vanschoren, J. (2019). *Automated machine learning: Methods, systems, challenges*. Springer Series on Challenges in Machine Learning.

Hutter, F., Lindauer, M., Balint, A., Bayless, S., Hoos, H., and Leyton-Brown, K. (2017). The configurable SAT solver challenge (CSSC). *Artificial Intelligence*, 243:1–25. 10.1016/j.artint.2016.09.006

Hutter, F., López-Ibáñez, M., Fawcett, C., Lindauer, M., Hoos, H., Leyton-Brown, K., and Stützle, T. (2014). AClib: A benchmark library for algorithm configuration. In *Learning and Intelligent Optimization*, volume 8426, pp. 36–40. Springer International Publishing. 10.1007/978-3-319-09584-4_4

Hutter, F., Xu, L., Hoos, H. H., and Leyton-Brown, K. (2014). Algorithm runtime prediction: Methods & evaluation. *Artificial Intelligence*, 206:79–111. 10.1016/j.artint.2013.10.003

Ishibuchi, H., Masuda, H., Tanigaki, Y., and Nojima, Y. (2015). Modified distance calculation in generational distance and inverted generational distance. In *Evolutionary Multi-Criterion*

Optimization, volume 9019, pp. 110–125. Springer International Publishing. 10.1007/978-3-319-15892-1_8

Karl, F., Pielok, T., Moosbauer, J., Pfisterer, F., Coors, S., Binder, M., Schneider, L., Thomas, J., Richter, J., Lang, M., Garrido-Merchán, E. C., Branke, J., and Bischl, B. (2023). Multi-objective hyperparameter optimization in machine learning—An overview. *ACM Transactions on Evolutionary Learning and Optimization*, 3(4):1–50. 10.1145/3610536

Knowles, J. (2006). ParEGO: A hybrid algorithm with on-line landscape approximation for expensive multiobjective optimization problems. *IEEE Transactions on Evolutionary Computation*, 10(1):50–66. 10.1109/TEVC.2005.851274

Koch, P., Wagner, T., Emmerich, M. T., Bäck, T., and Konen, W. (2015). Efficient multi-criteria optimization on noisy machine learning problems. *Applied Soft Computing*, 29:357–370. 10.1016/j.asoc.2015.01.005

Li, M., and Yao, X. (2020). Quality evaluation of solution sets in multiobjective optimisation: A survey. *ACM Computing Surveys*, 52(2):1–38.

Lindauer, M., Eggensperger, K., Feurer, M., Biedenkapp, A., and Deng, D. (2022). SMAC3: A versatile Bayesian optimization package for Hyperparameter Optimization. *Journal of Machine Learning Research*, 23(54):1–9.

López-Ibáñez, M., Dubois-Lacoste, J., Pérez Cáceres, L., Birattari, M., and Stützle, T. (2016). The irace package: Iterated racing for automatic algorithm configuration. *Operations Research Perspectives*, 3:43–58.

Mascia, F., López-Ibáñez, M., Dubois-Lacoste, J., and Stützle, T. (2014). Grammar-based generation of stochastic local search heuristics through automatic algorithm configuration tools. *Computers and Operations Research*, 51:190–199. 10.1016/j.cor.2014.05.020

Miranda, P., Silva, R., and Prudencio, R. (2015). I/S-Race: An iterative multi-objective racing algorithm for the SVM parameter selection problem. In *Annual Conference on Genetic and Evolutionary Computation 2022*.

Močkus, J. (1975). On Bayesian methods for seeking the extremum. In *Optimization Techniques IFIP Technical Conference Novosibirsk*, pp. 400–404.

Mockus, J., Tiesis, V., and Zilinskas, A. (1978). The application of Bayesian methods for seeking the extremum. *Towards Global Optimization*, 2:117–129.

Molnar, C., Casalicchio, G., and Bischl, B. (2019). Quantifying model complexity via functional decomposition for better post-hoc interpretability. In *Machine Learning and Knowledge Discovery in Databases - International Workshops of ECML PKDD*, volume 1167, pp. 193–204.

Ponweiser, W., Wagner, T., Biermann, D., and Vincze, M. (2008). Multiobjective optimization on a limited budget of evaluations using model-assisted S-metric selection. *Parallel Processing from Nature*, 5199:784–794.

Preuß, O. L., Rook, J., and Trautmann, H. (2024). On the potential of multi-objective automated algorithm configuration on multi-modal multi-objective optimisation problems. In S. Smith, J. Correia, and C. Cintrano (Eds.), *Applications of evolutionary computation*, pp. 305–321. Springer Nature Switzerland.

Realinho, V., Machado, J., Baptista, L., and Martins, M. (2021). Predict students' dropout and academic success (dataset).

Rook, J., Trautmann, H., Bossek, J., and Grimme, C. (2022). On the potential of automated algorithm configuration on multi-modal multi-objective optimization problems. In *Annual Conference on Genetic and Evolutionary Computation*, pp. 356–359.

Schede, E., Brandt, J., Tornede, A., Wever, M., Bengs, V., Hüllermeier, E., and Tierney, K. (2022). A survey of methods for automated algorithm configuration. *Journal of Artificial Intelligence Research*, 75:425–487. 10.1613/jair.1.13676

Thornton, C., Hutter, F., Hoos, H., and Leyton-Brown, K. (2013). Auto-weka: Combined selection and hyperparameter optimization of classification algorithms. In *Knowledge Discovery in Databases*, pp. 847–855.

Ugolotti, R., Sani, L., and Cagnoni, S. (2019). What can we learn from multi-objective meta-optimization of evolutionary algorithms in continuous domains? *Mathematics*, 7(3):232. 10.3390/math7030232

Vallati, M., Hutter, F., Chrpa, L., and McCluskey, T. (2015). On the effective configuration of planning domain models. In *International Joint Conference on Artificial Intelligence*, pp. 1704–1711.

Van Der Blom, K., Hoos, H. H., Luo, C., and Rook, J. G. (2022). Sparkle: Toward accessible meta-algorithmics for improving the state of the art in solving challenging problems. *IEEE Transactions on Evolutionary Computation*, 26(6):1351–1364. 10.1109/TEVC.2022.3215013

Yang, K., Emmerich, M., Deutz, A., and Fonseca, C. M. (2017). Computing 3-D expected hypervolume improvement and related integrals in asymptotically optimal time. In *Evolutionary Multi-Criterion Optimization*, volume 10173, pp. 685–700. 10.1007/978-3-319-54157-0_46

Zhang, T., Georgiopoulos, M., and Anagnostopoulos, G. (2013). S-Race: A multi-objective racing algorithm. In *Proceedings of the Fifteenth Annual Conference on Genetic and Evolutionary Computation Conference*, pp. 1565–1572.

Zhang, T., Georgiopoulos, M., and Anagnostopoulos, G. (2015). SPRINT multi-objective model racing. In *Proceedings of the 2015 Annual Conference on Genetic and Evolutionary Computation*, pp. 1383–1390.

Zitzler, E., and Thiele, L. (1998). Multiobjective optimization using evolutionary algorithms—A comparative case study. In *Parallel Problem Solving from Nature*, volume 1498, pp. 292–301.